

Feuille de travaux pratiques n° 2

Recherche récursive d'expressions régulières sur le Web

Code de départ associé : <https://github.com/DistributedComputing/TP-webgrep>.

1 Introduction

Pour pouvoir répondre efficacement aux requêtes des utilisateurs, les moteurs de recherche comme Google maintiennent un index de l'ensemble des pages du Web, accompagné de la liste des mots qu'elles contiennent. Cet index est construit grâce à des *robots d'indexation* (en anglais *web crawlers*) qui explorent automatiquement les pages du Web en suivant leurs hyperliens.

Le but de ce projet est d'implémenter un mini-robot d'indexation capable de rechercher des *expressions régulières* sur le Web. Par exemple, la commande suivante doit rechercher toutes les pages contenant le mot « Nantes », accessibles depuis la page Wikipédia de la ville :

```
$ java webgrep.WebGrep Nantes https://fr.wikipedia.org/wiki/Nantes
> https://fr.wikipedia.org/wiki/Nantes
> https://fr.wikiquote.org/wiki/Nantes
> https://fr.wiktionary.org/wiki/Nantes
> https://fr.wikivoyage.org/wiki/Nantes
> ...
```

2 Algorithme à implémenter

Pour obtenir les résultats, on commencera par chercher une occurrence de l'expression dans la page donnée en argument. Si la page contient l'expression, on continuera la recherche en suivant tous les liens de la page, récursivement. Le temps de recherche étant largement dominé par les accès réseau, on souhaite accélérer l'exploration en *parallélisant* le programme. Le but du TP est donc de concevoir une architecture parallèle efficace.

Une grande portion du code a déjà été faite pour vous. Téléchargez le code de départ dans le dépôt GitHub du TP. Une classe `Tools` est fournie en Java pour effectuer les tâches séquentielles. Elle contient principalement les méthodes statiques suivantes :

`Tools.initialize(args)` analyse les arguments du programme pour extraire l'expression régulière, les URL de départ et les options d'affichage. La liste complète des options peut être affichée en lançant le programme avec l'argument `"-help"`. Après initialisation, la liste des URL de départ est accessible via `Tools.startingURL()` et le nombre de threads à créer via `Tools.numberThreads()`.

`Tools.parsePage(address)` télécharge et analyse la page HTML à l'adresse indiquée, puis retourne un objet `ParsedPage` donnant accès à la liste des liens hypertextes (`page.hrefs()`) et à la liste des éléments HTML contenant l'expression régulière recherchée (`page.matches()`).

`Tools.print(page)` produit un affichage textuel de la `ParsedPage` passée en argument, en respectant les options d'affichage configurées lors de l'initialisation de `Tools`.

Remarque. le pare-feu des salles de TP bloque l'accès à Internet pour les requêtes automatiques. L'option `-o` (ou `--offline`), activée par défaut lorsque le programme est lancé sans argument, simule le chargement des pages sans se connecter à internet.

3 Architecture parallèle

L'architecture parallèle du programme doit respecter les contraintes suivantes :

1. **Parallélisation** : la recherche doit être répartie entre $n = \text{Tools.numberThreads}()$ threads. On créera un *pool* de n threads, auquel on soumettra une tâche pour chaque page à explorer.
2. **Exploration unique** : une même URL ne doit jamais être explorée deux fois. On conservera en mémoire la liste (ou l'ensemble) des pages déjà visitées. La vérification qu'une page n'y figure pas et son insertion doivent être effectuées dans un même bloc atomique (pourquoi?). On pourra pour cela utiliser la classe `ConcurrentSkipListSet` et, en particulier, sa méthode `add`.
3. **Sortie cohérente** : il ne faut pas d'entrelacement entre les sorties de deux threads. La méthode statique `Tools.print` fournie n'est pas thread-safe. On suggère de dédier un thread unique aux sorties textuelles, selon le modèle *producteur-consommateur* : les producteurs sont les threads du pool, qui produisent des objets `ParsedPage`, et le consommateur est un thread séparé chargé de l'affichage.